

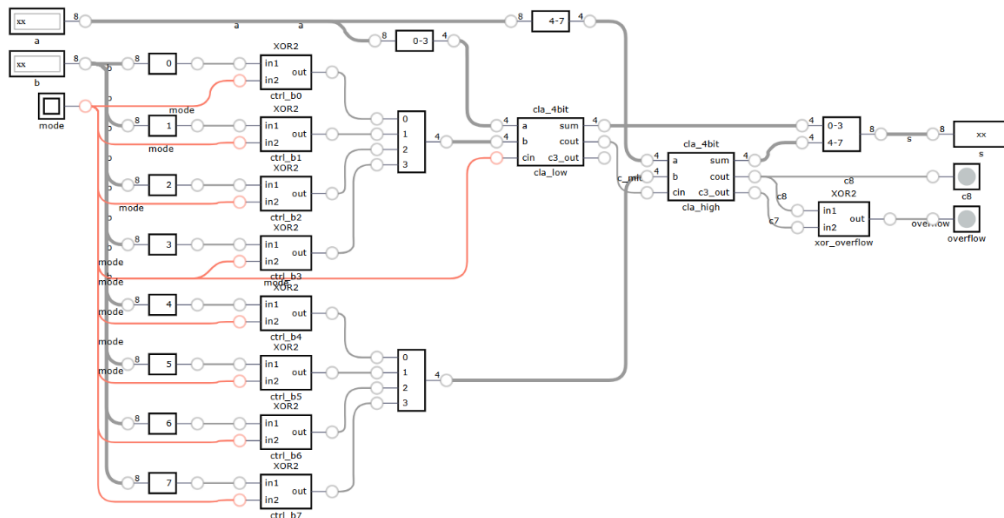
Logic Design HW1:

8-bit Carry-Lookahead Adder (CLA)

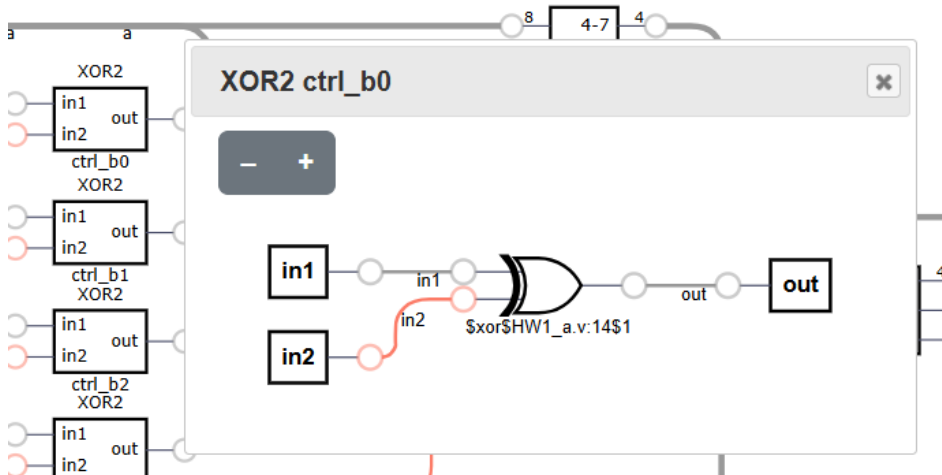
Name : 魏彤凌

Student ID : 114010005

1. Block diagrams for each module

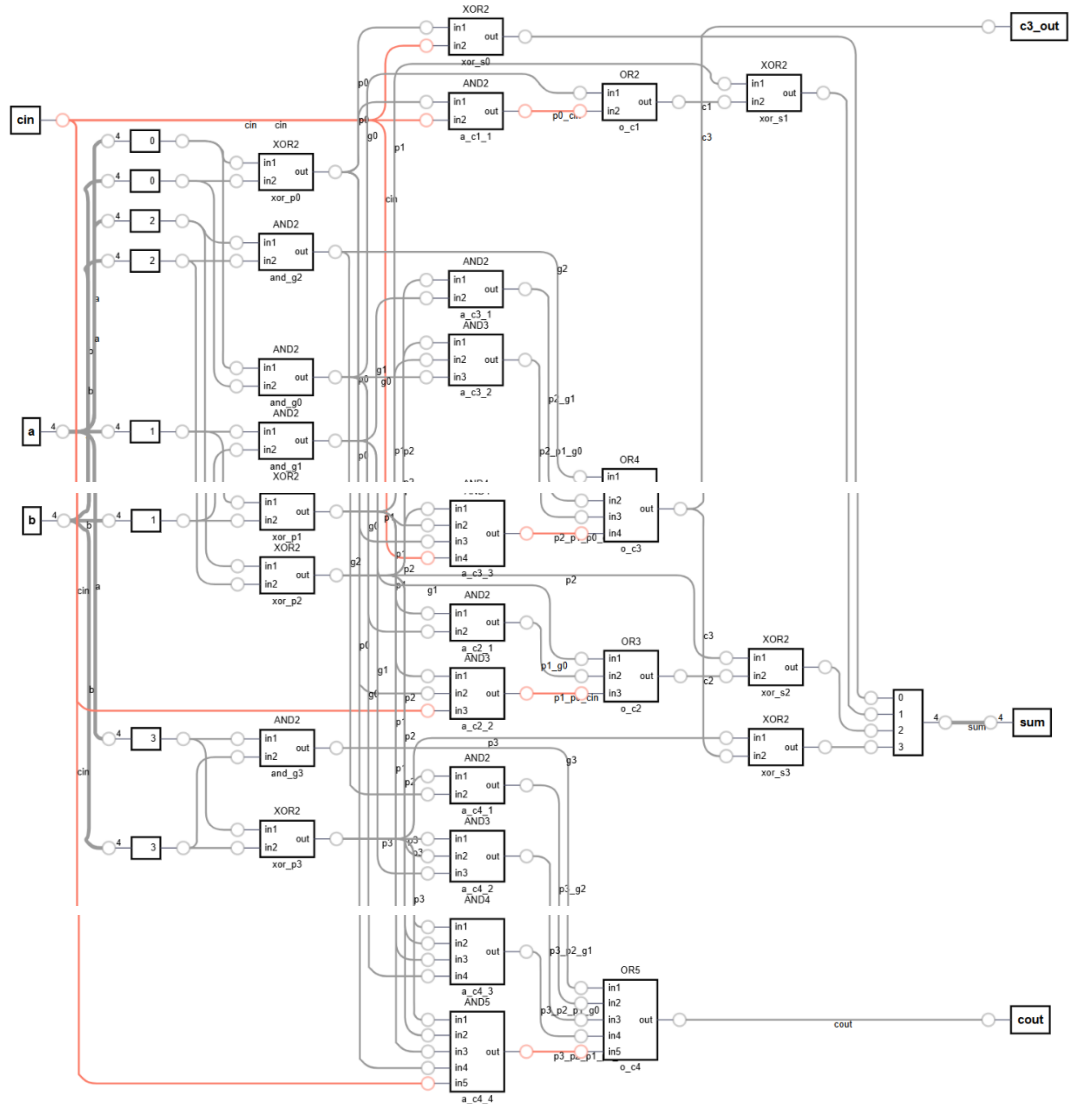


圖一：CLA_AddSub_8bit 模組



圖二：一個邏輯閘（以 XOR2 ctrl_b0 為例）

cla_4bit cla_low



圖三：:4 位元超前進位加法器（以 cla_4bit cla_low 為例）

詳見網址：<https://reurl.cc/OQk5lX>

2. Implementation strategy for each module

整體電路架構拆解為三個階層：

- 基礎邏輯閘層 (Primitive Gate Layer)：包裝基本的邏輯閘 (AND2～AND5、OR2～OR5、XOR2)。
- 子模組層 (Sub-module Layer)：4 位元超前進位加法器 (cla_4bit)。
- 頂層模組 (Top Level Module)：8 位元超前進位加減法器 (CLA_AddSub_8bit)。

4 位元超前進位加法器 (cla_4bit) 使用 AND2 與 XOR2 閘計算每位元的 $G_i = A_i \cdot B_i$ 與 $P_i = A_i \oplus B_i$ 。再使用 AND2～AND5 與 OR2～OR5 閘將 SOP 邏輯公式實作出來。特別增設 c3_out 輸出引腳，將模組內部的第 3 位進位 C_3 導出。這是為了讓頂層模組能抓到進到最高位元的進位，進而計算有號數溢位。

8 位元超前進位加減法器 (CLA_AddSub_8bit) 是利用兩個串聯的 4-bit CLA 模組 (cla_low 與 cla_high)，避免直接撰寫 8-bit CLA 公式，導致邏輯閘輸入端過大、硬體電路過於複雜。利用 mode 控制輸入端 B 經過一組 XOR2 陣列，達成第二補數的轉換。使用 XOR2 閘計算 $\text{Overflow} = C_8 \oplus C_7$ 。其中 C_8 為最終的 c8 (cout)，而 C_7 則來自 cla_high 輸出的 c3_out。

3. Carry-lookahead theory and justification for using it

超前進位 (CLA) 公式：

$$\text{傳播項：} P_i = A_i \oplus B_i$$

$$\text{生成項：} G_i = A_i \cdot B_i$$

$$C_1 = G_0 + P_0 \cdot C_0$$

$$C_2 = G_1 + P_1 \cdot C_1 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_0$$

$$C_3 = G_2 + P_2 \cdot C_2 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

$$C_4 = G_3 + P_3 \cdot C_3$$

$$= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_0$$

使用理由：漣波進位 (RCA) 的延遲會隨位元數成線性增加 $O(N)$ ；而超前進位 (CLA) 能平行計算所有進位，將傳遞延遲降至最低 $O(1)$ ，大幅提升計算效能。

4. How subtraction is performed via two's complement using the mode input and the initial carry.

Two's complement:

$$A - B = A + (\sim B) + 1$$

當 mode = 0 (加法)：

輸入 B 經過 XOR 閘後維持原值 B ($B \oplus 0 = B$)，且初始進位 $C_{in} = 0$ ，
電路執行

$$A + B + 0$$

當 mode = 1 (減法)：

輸入 B 經過 XOR 閘後全部反相成 $\sim B$ ($B \oplus 1 = \sim B$)，同時將
mode = 1 接到低位 CLA 的初始進位 C_{in} 提供「+1」，電路成功執行

$$A + \sim B + 1$$

5. How the signed overflow flag is derived and why the internal carry c_7 had to be exposed.

溢位發生於最高有效位元的進位輸入與進位輸出不一致時，公式為

$$Overflow = C_8 \oplus C_7$$

一般 4 位元的 CLA 只會輸出最高位的進位 C_4 。但在高位的 4 位元 CLA 中，進到第 7 位元的進位（全域的 C_7 ）屬於模組內部的訊號。Verilog 不允許外部模組直接存取子模組內部的線路，因此我們必須在 cla_4bit 專門增加一個輸出引腳（c3_out），將這個內部訊號引出給 Top Module，才能順利做 $C_8 \oplus C_7$ 的溢位判斷。

6. Testbench strategy

正數加法：無進位/無溢位（如 $5 + 3 = 8$ ）。

正數減法：結果為正數（如 $10 - 4 = 6$ ）。

加法溢位（Overflow）：正數加正數變成負數（如 $127 + 1 = -128$ ）。

減法溢位（Overflow）：正數減負數變成負數（如 $127 - (-1)$ ）。

負數減法：大數減小數（如 $5 - 12 = -7$ ）。

使用了#10 延遲留時間給電路穩定，並使用 monitor 與 dumpfile 來記錄輸出與波形。

7. Summary of simulation results, including any notable observations.

所有測試案例的輸出結果（S, C_8 , Overflow）皆與理論值完全吻合，證實電路邏輯正確。

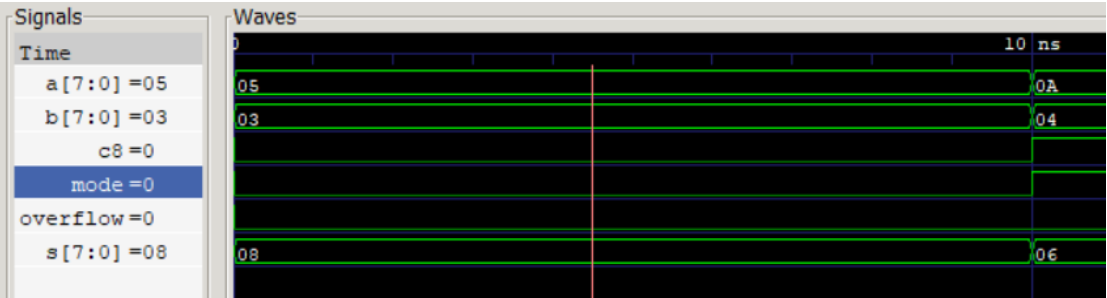
值得注意的小細節：

- 觀察到在無符號數與有號數下， C_8 （Carry-out）與 Overflow 的意義不同：例如 $127 + 1$ 時， $C_8 = 0$ （因為沒有超過 255），但 $Overflow = 1$ （因為超過了有號數的 127）。
- 觀察到減法時（ $mode = 1$ ）， $C_8 = 1$ 其實代表沒有借位，而 $C_8 = 0$ 則代表發生了借位。

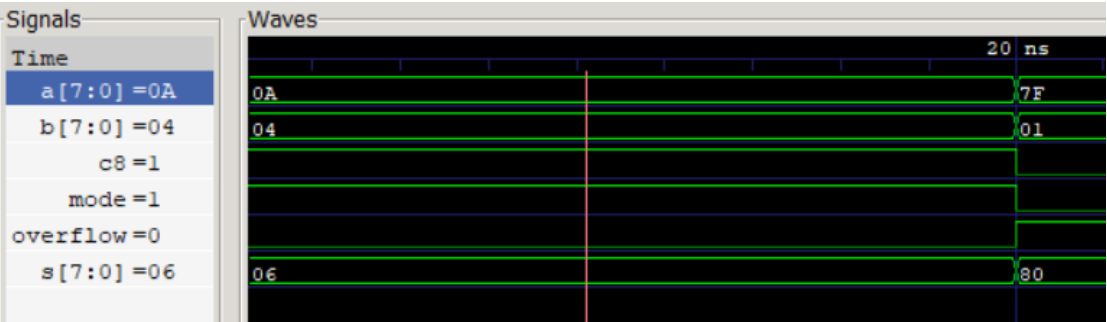
8. Screenshot of the simulation waveform showing inputs, mode, sum/difference, carry-out, and overflow.

```
Time=0ns | mode=0 | a= 5 (05) | b= 3 (03) | s= 8 (08) | c8=0 | overflow=0
Time=10ns | mode=1 | a= 10 (0a) | b= 4 (04) | s= 6 (06) | c8=1 | overflow=0
Time=20ns | mode=0 | a=127 (7f) | b= 1 (01) | s=128 (80) | c8=0 | overflow=1
Time=30ns | mode=1 | a=127 (7f) | b=255 (ff) | s=128 (80) | c8=0 | overflow=1
Time=40ns | mode=1 | a= 5 (05) | b= 12 (0c) | s=249 (f9) | c8=0 | overflow=0
finish testing!
```

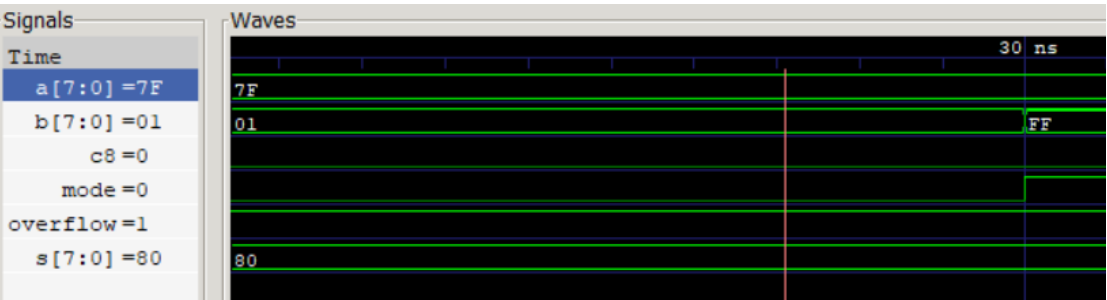
圖四：終端機截圖



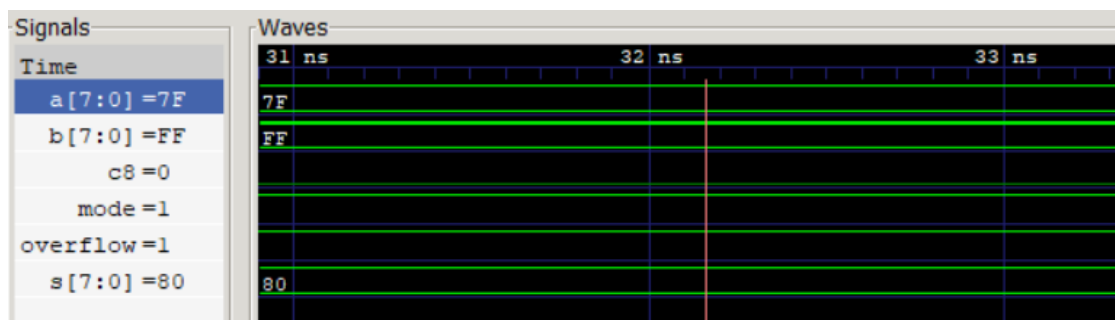
圖五：測試一



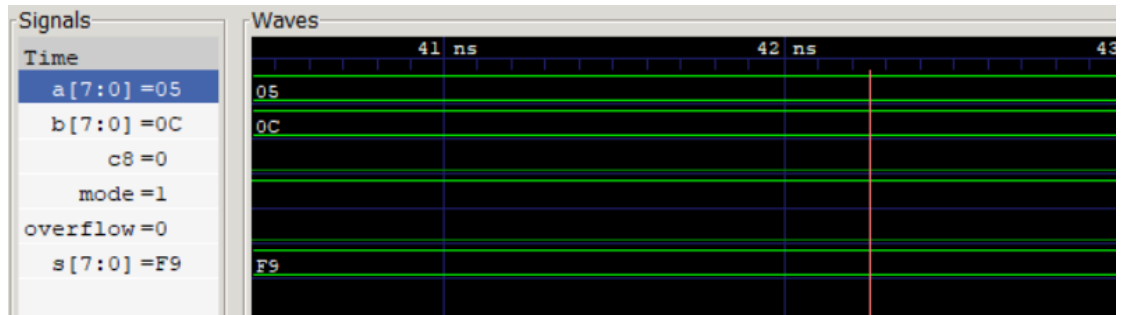
圖六：測試二



圖七：測試三



圖八：測試四



圖九：測試五